

Learn

This part of the documentation teaches DCGM concepts and workflows. It is organized for readers who are learning how DCGM fits together or how to complete an operational task. For exhaustive command syntax, accepted values, defaults, output, and exit behavior, use the [Command Line](#). For programmatic interfaces, use the [C/C++ API](#). Product scope, focus areas, and audiences are on the top-level [About](#) page.

NVIDIA Data Center GPU Manager (DCGM) is a host-level framework for inventory, telemetry, health, policy, configuration, diagnostics, profiling, and workload accounting on NVIDIA data-center hardware.

For supported platforms, packages, service startup, and installation verification, see [Install DCGM](#).

How DCGM Fits

DCGM uses the low-level NVIDIA management software appropriate to each device and presents the results through one host-level model. Drivers and platform services still control the hardware. On NVSwitch systems, Fabric Manager remains responsible for fabric configuration and lifecycle.



DCGM's position in the management stack

The host engine represents managed objects as *entities*. Physical GPUs, MIG instances, NVSwitches, NVLink ports, supported NVIDIA CPUs and cores, and supported ConnectX devices are different entity families.

Inventory values, gauges, counters, and events are represented as typed *fields*. A *watch* asks the host engine to collect selected fields for selected entities at a requested cadence and retain the resulting samples for a bounded period.

The host engine also owns:

- entity groups and field groups that provide reusable selections;
- cached field samples with collection timestamps and statuses;
- module state for health, policy, configuration, diagnostics, profiling, and platform-specific features; and
- process watches and job records.

A field read normally returns a retained sample rather than forcing a new hardware query. Use the sample timestamp and status when interpreting it. Groups select entities; they do not allocate, isolate, or grant access to hardware. Host-engine groups, watches, samples, and job records are runtime state and normally do not survive an engine restart.

Continue with [Core Services](#) for the complete entity, field, watch, and lifetime model, or select a role-specific path below.

Learning Path

Continue by role

[Getting Started for System Administrators](#)

Understand the long-lived host engine as a shared management plane, from runtime inventory and sampled telemetry through authority, restarts, and DCGM Exporter deployment.

[Getting Started for Application Developers](#)

Understand DCGM’s stateful shared collection and choose standalone or embedded host-engine operation.

Functionality by module

DCGM separates its always-loaded core services from loadable feature modules. This organization lets readers approach DCGM by the same broad boundaries used by the implementation.

Product-class availability is summarized in [Feature Availability by GPU Class](#). For module loading, state, and denylisting, see [Modules](#). For exact interfaces, use the [Command Line](#) or [C/C++ API](#).

Functionality areas	
Area	Functionality
Core Services	Entity discovery and groups, field watching, process and job statistics, topology, and link state. Core is always loaded.
Modules	Learning pages for the ten loadable modules: NVSwitch and ConnectX, vGPU, introspection, health, policy, configuration, diagnostics, profiling, CPU monitoring, and multi-node diagnostics.

Cross-cutting workflows

Detached GPUs

Understand dynamic driver and GPU attachment and the effect of detached entities on DCGM features.

Error Injection

Exercise monitoring, health, and policy responses with synthetic field values in an isolated test environment.

Operations and troubleshooting

Debugging and Troubleshooting

Collect useful logs and diagnostic evidence when investigating a problem.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.